

子问题 1 求解算法——初稿

1. 问题分析

子问题 1 要求在无产量约束条件下最大化原材料体积利用率，等价于最小化废料体积。从数学模型角度看，这是一个 0/1 整数规划问题，决策变量 $place_{b,j,k}$ 为 0 或 1， $ori_{b,j,k}$ 有 6 种取值， (x, y, z) 为连续变量。不重叠约束是析取约束，导致问题呈组合爆炸特性——精确算法（如 MILP 分支定界）在 15 块原材料、7 种工件、6 种姿态的规模下计算不可行。

因此采用 **Empty Maximal Spaces (EMS)** 构造启发式算法——一种专门面向 3D 矩形装箱的贪心方法，利用空间分裂策略隐式保证约束满足，避免显示枚举所有组合。

2. 算法核心思想

EMS 算法维护一个**互不重叠的空闲空间列表**，每一步从其中选择一个空间放置工件，放置后更新列表。空间列表始终覆盖所有尚未使用的体积，且空间之间两两不相交，因此后续工件只要放入某个空间，就自动满足边界约束与不重叠约束。

一轮迭代的三个步骤：

- Best-Fit 选择** — 对所有候选工件与所有空闲空间的组合，计算贴合分数 $score = (sp.dx - dx) + (sp.dy - dy) + (sp.dz - dz)$ ，选分数最小者。分数越小，工件放入后三轴总间隙越小，贴合越紧密。
- 空间分裂** — 将选中工件紧贴空间原点放置后，被占用空间按六个方向拆分为子空间（下/上/前/后/左/右），仅保留尺寸为正者。
- 相交切除与合并** — 新工件可能横向延伸到其他空间区域，遍历所有空间检查交集，对存在交集者执行同样的分裂切除操作。随后扫描相邻空间，若两空间共享完整面且另两轴对齐则合并为一，减少碎片化。

三步执行完毕后，空间列表恢复「互不重叠 + 完全覆盖剩余空域」的性质，算法进入下一轮迭代。当所有空间都无法容纳任何候选工件时，该原材料块打包完成。

3. 数据结构

3.1 空闲空间 Space

Space:

```
x, y, z    # 空间原点坐标
dx, dy, dz # 三轴方向长度
```

一个 Space 代表原材料块内部的一个矩形可用区域。尺寸均为整数。

3.2 工件实例

PlacedItem:

```
type      # 工件型号 (含姿态信息, 如 "J05_60x80x50")
x, y, z    # 放置位置 (紧贴空间原点)
dx, dy, dz # 实际占用尺寸 (经姿态旋转后的长宽高)
```

4. 伪代码

```
function pack(block, candidates):
    spaces = [Space(0, 0, 0, block.L, block.W, block.H)]
    placed = []

    while candidates not empty:
        # 第一步: Best-Fit 选择
        best_score = inf
        for each item in candidates:
            for each sp in spaces:
                if sp.can_fit(item.dx, item.dy, item.dz):
                    score = (sp.dx-item.dx)+(sp.dy-item.dy)+(sp.dz-item.dz)
                    if score < best_score:
                        best_score = score
                        best = (item, sp)

        if best_score == inf: break # 无可放置的工件

        # 第二步: 放置工件, 分裂空间
        item, sp = best
        placed.append(item at (sp.x, sp.y, sp.z))
        candidates.remove(item)
        spaces.remove(sp)
        fragments = split_by_removal(sp, sp.x, sp.y, sp.z, item.dx, item.dy, item.dz)
        spaces.extend(fragments)
```

```

# 第三步：切除相交
item_box = Space(sp.x, sp.y, sp.z, item.dx, item.dy, item.dz)
new_spaces = []
for each s in spaces:
    inter = s.intersection(item_box)
    if inter is None:
        new_spaces.append(s)
    else:
        fragments = split_by_removal(s, inter)
        new_spaces.extend(fragments)
spaces = new_spaces

# 合并相邻空间
spaces = try_merge(spaces)

return placed

```

4.1 split_by_removal(sp, occupied) — 空间分裂

从空间 sp 中切除已占用区域 (x, y, z, dx, dy, dz) ，沿六个方向生成子空间：

方向	新空间区域
下方	$(sp.x, sp.y, sp.z, sp.dx, sp.dy, z - sp.z)$
上方	$(sp.x, sp.y, z + dz, sp.dx, sp.dy, sp.z + sp.dz - (z + dz))$
前方	$(sp.x, sp.y, z, sp.dx, y - sp.y, dz)$
后方	$(sp.x, y + dy, z, sp.dx, sp.y + sp.dy - (y + dy), dz)$
左方	$(sp.x, y, z, x - sp.x, dy, dz)$
右方	$(x + dx, y, z, sp.x + sp.dx - (x + dx), dy, dz)$

仅保留 $dx > 0 \wedge dy > 0 \wedge dz > 0$ 的区域。

4.2 try_merge(spaces) — 空间合并

两空间 a, b 可沿 x 轴合并当且仅当：

- $a.x + a.dx = b.x$ (a 的右面与 b 的左面重合)
- $a.y = b.y, a.z = b.z$ (y 轴和 z 轴起点对齐)
- $a.dy = b.dy, a.dz = b.dz$ (y 轴和 z 轴长度一致)

合并为： $Space(a.x, a.y, a.z, a.dx + b.dx, a.dy, a.dz)$

y 轴、 z 轴方向的合并规则对称可推。

5. 多策略优化

同一批候选工件，排序不同会导致贪心路径不同，最终打包效果可能有显著差异。因此对每块原材料独立尝试三种排序策略：

策略	排序规则	直觉
体积优先	按工件体积降序	大件先入，小件填空隙
底面积优先	按 $dx \times dy$ 降序	先铺底面，逐层向上
最长边优先	按 $\max(dx, dy, dz)$ 降序	最长的先定位

每块原材料分别以三种策略运行 `pack()`，选取所用体积最大者作为该块的最终方案。由于子问题 1 无产量约束，每块拥有独立的候选池（每种工件 $\times$ 姿态复制 80 份），互不影响。

6. 复杂度分析

设候选工件数为 n （本问题中 $n \approx 7 \times 6 \times 80 = 3360$ ），空闲空间数最多为 $O(m)$ ，其中 m 为已放置工件数。每轮迭代需扫描 $O(n \cdot m)$ 个组合，共约 m 轮，总时间复杂度为 $O(n \cdot m^2)$ 。

对于本问题，每块约放置 $15 \sim 48$ 件， m 很小，因此单块运行时间在秒级以内。15 块 $\times$ 3 种策略均在数分钟内完成。

7. 求解结果

总体指标

指标	数值
原材料总体积	75,750,000 mm ³
已使用体积	74,450,000 mm ³
废料体积	1,300,000 mm ³

指标	数值
体积利用率	98.28%
总工件数	405 件

各块详情

原材料块	工件数	利用率	主要工件组合
L01 × 5	48	100.00%	J05 × 30, J06 × 18
L02 × 5	18	94.93%	J04 × 8, J05 × 8, J06 × 2
L03 × 5	15	97.08%	J05 × 6, J06 × 8, J03 × 1

工件生产数量

工件	数量	总体积
J03	5	450,000
J04	40	7,200,000
J05	220	52,800,000
J06	140	14,000,000
J01, J02, J07	0	0

结果分析

L01 块 ($300 \times 200 \times 150$) 的 100% 利用率源于数学上的完美平铺：底面积优先策略将 J05 的 $60 \times 80 \times 50$ 姿态沿 x 方向排布 5 件 ($60 \times 5 = 300$)，沿 y 方向排布 2 排 ($80 \times 2 = 160$)，沿 z 方向 3 层 ($50 \times 3 = 150$)，剩余 y 方向 40 的空隙恰好由 J06 的 $100 \times 20 \times 50$ 姿态以 $3 \times 2 \times 3$ 的排列填满。

需要说明的是，由于子问题 1 没有产量约束，算法倾向于反复使用少数几种体积较大且适配性好的工件（J05 占总体积的 70.9%），导致 J01、J02、J07 产量为 0。若实际生产中要求每种工件至少有一定产量，需转入子问题 2 的约束模型。